

Lab 4 Report

Mobile Application Development

Lab	Lab 4
Topic	Real-time 2D mobile game development
Language	Kotlin
Member	Nguyễn An Đức – 22520268 Đoàn Hoài Việt - 22521662

1. Overview

For this lab, I built a small 2D shooting game inspired by the classic chicken shooting style game. The player controls a spaceship at the bottom of the screen and fires bullets upward to destroy enemies. The game is drawn manually on a Canvas, so the screen has to be updated continuously instead of relying on normal Android widgets.

The project uses SurfaceView for drawing, GameThread for the main loop, and separate classes for the player bullets, enemies, boss, score state, rendering, and game logic. I split the code this way so the main GameView does not become too crowded.

2. Source Code Organization

File	What it does
MainActivity.kt	Starts the game by setting GameView as the activity content.
GameThread.kt	Runs the update and draw loop at around 60 frames per second.
GameView.kt	Handles SurfaceView lifecycle and touch input.
GameState.kt	Stores score, high score, lives, speeds, bullets, enemies, and reset data.
GameWorld.kt	Updates game objects, checks collisions, and controls enemy or boss spawning.
GameRenderer.kt	Draws the background, player, HUD, boss, enemies, bullets, and game over screen.
BulletFactory.kt	Creates bullet patterns for different upgrade levels.
FiringObject.kt	Represents a bullet and its collision rectangle.
Opponent.kt	Represents a normal enemy with movement, health, and a health bar.

Boss.kt	Represents the larger boss enemy that moves left and right.
GameManager.kt	Creates bitmap graphics and enemy objects.

3. Main Game Loop

The game loop is placed in GameThread. Each frame locks the Canvas, updates the game state, draws the new frame, and then unlocks the Canvas. A short sleep is used when a frame finishes early, which keeps the game speed more stable and avoids wasting CPU.

```

override fun run() {
    val frameTime = (1000 / targetFPS).toLong()

    while (running) {
        val startTime = System.nanoTime()
        var canvas: Canvas? = null

        try {
            canvas = surfaceHolder.lockCanvas()
            if (canvas != null) {
                synchronized(surfaceHolder) {
                    gameView.update()
                    gameView.draw(canvas)
                }
            }
        } finally {
            if (canvas != null) {
                surfaceHolder.unlockCanvasAndPost(canvas)
            }
        }

        val timeMillis = (System.nanoTime() - startTime) / 1_000_000
        val waitTime = frameTime - timeMillis
        if (waitTime > 0) sleep(waitTime)
    }
}

```

4. Exercise Implementation

Exercise 1 - Highest score display

I added a high score value to GameState and saved it with SharedPreferences. Every time the current score passes the old high score, the value is updated immediately. This makes the high score remain available even after restarting the game.

```

var highScore = preferences.getInt("high_score", 0)
private set

fun addScore(points: Int) {
    score += points
    if (score > highScore) {

```

```

        highScore = score
        saveHighScore()
    }
}

private fun saveHighScore() {
    preferences.edit().putInt("high_score", highScore).apply()
}

canvas.drawText("High: ${state.highScore}", 28f, 108f, textPaint)

```

Exercise 2 - Three lives for each session

The player starts with three lives. An enemy reaching the bottom boundary no longer ends the game immediately. Instead, the game removes that enemy and subtracts one life. Game over only happens when the life count reaches zero.

```

var lives = 3
var gameOver = false

fun loseLives(amount: Int) {
    lives = (lives - amount).coerceAtLeast(0)
    if (lives == 0) {
        gameOver = true
        saveHighScore()
    }
}

val escapedOpponents = state.opponents.filter {
    it.reachedBoundary(screenHeight) || it.isOffScreen(screenHeight)
}

if (escapedOpponents.isNotEmpty()) {
    state.opponents.removeAll(escapedOpponents.toSet())
    state.loseLives(escapedOpponents.size)
}

```

Exercise 3 - Spaceship at the shooting position

The player ship is drawn near the bottom of the screen. When the user touches the screen, I update the ship's X position first and then fire bullets from that location. The X value is clamped so the ship does not move outside the screen.

```

override fun onTouchEvent(event: MotionEvent): Boolean {
    if (event.action == MotionEvent.ACTION_DOWN) {
        synchronized(holder) {
            if (state.gameOver) {
                state.reset(width, height)
                return true
            }

            state.playerX = event.x.coerceIn(48f, width - 48f)
            fireBullets()
        }
    }
    return true
}

```

```

        }
        return true
    }
    return true
}

private fun drawPlayer(canvas: Canvas) {
    val bitmap = gameManager.playerBitmap
    val left = state.playerX - bitmap.width / 2f
    val top = state.playerY - bitmap.height / 2f
    canvas.drawBitmap(bitmap, left, top, null)
}

```

Exercise 4 - Bullet spread and upgrade levels

Bullet upgrades are based on the score. At the beginning, the player fires one bullet. As the score increases, the level increases and the ship fires more bullets with different angles. The horizontal and vertical bullet speeds are calculated from the angle.

```

fun bulletLevel(): Int {
    return (score / 50 + 1).coerceIn(1, 5)
}

private fun spreadDegrees(level: Int): List<Float> {
    return when (level) {
        1 -> listOf(0f)
        2 -> listOf(-8f, 8f)
        3 -> listOf(-14f, 0f, 14f)
        4 -> listOf(-18f, -6f, 6f, 18f)
        else -> listOf(-24f, -12f, 0f, 12f, 24f)
    }
}

val radians = Math.toRadians((270f + degrees).toDouble())
val velocityX = (cos(radians) * speed).toFloat()
val velocityY = (sin(radians) * speed).toFloat()
FiringObject(centerX, startY, velocityX, velocityY, damage = 1)

```

Exercise 5 - Enemy health bar

Each enemy receives a random amount of health when it is created. A bullet hit reduces the health by the bullet damage. The enemy is removed only when its health becomes zero. I also draw a small health bar above the enemy so the player can see how much health is left.

```

val health = Random.nextInt(2, 7)
return Opponent(x, y, speed, bitmap, laneVelocity, health)

fun takeDamage(amount: Int): Boolean {
    health = (health - amount).coerceAtLeast(0)
    return health == 0
}

private fun drawHealthBar(canvas: Canvas) {
    val barHeight = 8f

```

```

val top = y - 14f
canvas.drawRect(x, top, x + width, top + barHeight, healthBarBackground)

val healthRatio = health.toFloat() / maxHealth.toFloat()
canvas.drawRect(x, top, x + width * healthRatio, top + barHeight, healthBarPaint)
}

```

Exercise 6 - Lane-based enemy movement

To make enemy movement less predictable, I gave each enemy a horizontal lane velocity. Some enemies move left, some move right, and some still move straight down. When an enemy touches the side of the screen, its horizontal direction is reversed.

```

val laneVelocity = listOf(-3.5f, -2f, 0f, 2f, 3.5f).random()

fun update(screenWidth: Int) {
    y += speed
    x += laneVelocity

    if (x <= 0f || x + width >= screenWidth) {
        laneVelocity *= -1f
        x = x.coerceIn(0f, screenWidth - width)
    }

    updateRect()
}

```

Exercise 7 - Boss enemy that spawns smaller enemies

The boss appears after the player has earned enough score. It stays near the top of the screen and moves horizontally. Instead of normal random spawning, the boss periodically creates smaller enemies from its center position. This makes the later part of the game feel different from the early part.

```

if (state.score >= 60 && state.bossRespawnCounter > 240) {
    state.bossRespawnCounter = 0
    state.boss = Boss(
        (screenWidth - gameManager.bossBitmap.width) / 2f,
        80f,
        gameManager.bossBitmap
    )
    return
}

fun update(screenWidth: Int): Boolean {
    x += direction * 6f
    if (x <= 0f || x + width >= screenWidth) {
        direction *= -1f
        x = x.coerceIn(0f, screenWidth - width)
    }

    spawnCooldown++
    if (spawnCooldown >= 55) {
        spawnCooldown = 0
    }
}

```

```
        return true
    }
    return false
}

if (currentBoss.update(screenWidth)) {
    state.opponents.add(
        gameManager.createOpponent(
            currentBoss.spawnX(),
            currentBoss.spawnY(),
            state.opponentBaseSpeed + 1.5f
        )
    )
}
```

5. Gameplay Flow

- MainActivity opens GameView as the game screen.
- When the SurfaceView is ready, GameThread starts the continuous update and draw loop.
- GameWorld moves bullets, enemies, and the boss, then checks collisions and boundary conditions.
- GameRenderer draws the current frame, including the HUD and game over message.
- Touch input moves the spaceship and creates bullets through BulletFactory.